DataTeach.ai

PYTHON BASICS EBOOK

A Friendly Beginner's Guide

www.datateach.ai

Introduction

Most beginners don't fail because Python is hard. They fail because they ***don't see the bigger picture.***

This guide is different.

You won't just learn syntax. You'll understand:

- Why things work
- When to use them
- How they connect

What is Python?

Python is a high-level, easy-to-learn programming language used to build software, analyze data, automate tasks, and create intelligent systems.

In simple words:
Python helps you talk to computers in a simple and readable way.

Python is one of the most popular programming languages because:

- Simple and easy to understand
- Less code compared to other languages
- Huge community support
- Used in real world applications

Your First Python code

```
print("Hello, World!")
```

Chapter 1: Keywords

Definition

Keywords are ***reserved words in Python that already have a fixed meaning and purpose.***

You cannot change or reuse them.

Why Keywords Matter

Keywords control the flow and logic of your program. Without them, Python cannot make decisions.

Example:

```
age = 20

if age>18:
   print(“Adult”)
```

Breakdown

- if → decision-making keyword
- > → comparison operator
- condition → age > 18

Python reads it like: "If this is true, do this"

Common Mistake

Trying to rename keywords

```
if = 10  # not allowed
```

Keywords are like traffic signals

- **if** → check condition
- **else** → alternative path

Chapter 2: Identifiers

What are Identifiers?
Identifiers are the names you give to variables, functions, or objects.

Simply: Names in your program

Rules for Identifiers

- Must start with a letter or underscore _
- Cannot start with a number
- Cannot use keywords
- Can contain letters, numbers, underscore

Examples

```
name = "Satya"
age = 25
_total = 100
```

Invalid Example:

```
1name = “satya”
for = 10
```

Best Practices

✔️ Use meaningful names
✔️ Keep it readable

- **Good:** student_name = "John"
- **Bad:** x = "John"

Chapter 3: Variables

What is a Variable?
A variable is a named location in memory used to store data.

In simple words:
A variable is a container that holds a value.

How to Create a Variable

```
name = "Nabeela"
age = 22
```

Creating a variable is called assignment, where a value is assigned to a name using the = operator.

Variable Naming Rules

A variable name:

- Must start with a letter (a-z, A-Z) or underscore _
- Cannot start with a number
- Can contain letters, numbers, and underscores
- Is case-sensitive (name and Name are different)

```
valid_name = "Python"
_age = 22

# Invalid
# 1name = "Error"
```

Example:

```
x = 10
y = 5

print (x+y)  #15
```

Definition:
Variables store values that can be used, changed, and reused in a program

Dynamic Typing

```
value = 10
value = "Hello"
```

Definition:
Python is dynamically typed, meaning a variable can change its data type during execution.

Multiple Assignment

```
a, b, c = 1, 2, 3
```

You can assign multiple values to multiple variables in one line.

Real Example:

```
name = Nabeela
score = 95
print(name, “scored”, score)
```

Variables do not store data directly; they refer to values stored in memory.

Keywords vs Identifiers

Reserved words in a programming language	Names given to variables, functions, classes
Define syntax and structure	Identify elements in the program
Fixed (cannot be changed)	Flexible (you can create any valid name)
Cannot be used as names	Cannot use keywords as identifiers
if, else, while, class	age, total_sum, myFunction

Variables vs Keywords

Named storage used to hold data	Reserved words in a programming language
Store and manipulate values	Define syntax and control flow
Flexible (names can be created)	Fixed (cannot be changed)
Holds a value (e.g., number, string)	Cannot be used as variable names
Must follow naming rules	if, else, for, while

Data Types in Python

Definition
A data type defines the kind of value a variable can store and the operations that can be performed on it.

In simple words:
Data types tell Python what type of data you are using (number, text, etc.).

Common Data Types in Python

1. Numeric Types
Numeric data types are used to store numbers.

```
x = 10       # int (integer)
y = 3.14     # float (decimal number)
z = 2 + 3j   # complex (complex number)
```

- **int** → Whole numbers
- **float** → Decimal numbers
- **complex** → Numbers with real and imaginary parts

2. String (str)
A string is a data type used to store text (sequence of characters).

```
name = "Nabeela”
```

3. Boolean (bool)
Boolean data type represents logical values.

```
is_active = True
is_admin = False
```

Only two values: **True** or **False**

4. List
A list is an ordered, mutable collection of items.

```
numbers = [1, 2, 3, 4]
```

Allows duplicates
Can be changed (mutable)

5. Tuple
A tuple is an ordered, immutable collection of items.

```
coordinates = (10, 20)
```

Cannot be changed after creation

6. Set
A set is an unordered collection of unique elements.

```
unique_numbers = {1, 2, 3}
```

No duplicates allowed

7. Dictionary (dict)

Each key maps to a value

```
student = {"name": "Satya", "age": 25}
```

Each key maps to a value

Checking Data Type

```
x = 10
print(type(x)) # <class 'int'>
```

type( ) is used to identify the data type of a variable.

Data types are essential because they determine how data is stored, processed, and used in a program.

Operations in Python

Operations are actions performed on values (data) using operators to produce a result.

iIn simple words:
Operations are calculations or actions you perform on data.

Types of Operations

1. Arithmetic Operations
Definition:
Used to perform mathematical calculations.

```
a = 10
b = 3

print(a + b)  # Addition → 13
print(a - b)  # Subtraction → 7
print(a * b)  # Multiplication → 30
print(a / b)  # Division → 3.33
print(a % b)  # Modulus (remainder) → 1
print(a ** b) # Power → 1000
print(a // b) # Floor division → 3
```

2. Comparison Operations

Definition:
Used to compare two values and return a boolean (True or False).

```
a = 10
b = 5

print(a > b)       # True
print(a < b)       # False
print(a == b)    # False
print(a != b)     # True
```

3. Logical Operations

Definition:
Used to combine multiple conditions.

```
x = True
y = False

print(x and y)  # False
print(x or y)     # True
print(not x)     # False
```

4. Assignment Operations

Definition:
Used to assign and update values in variables.

```
x = 10
x += 5   # x = x + 5
x -= 2   # x = x - 2
```

5. Membership Operations

Used to check if a value exists in a sequence (like string, list).

```
text = "Python"

print("P" in text)        # True
print("z" not in text)  # True
```

6. Identity Operations

Used to check if two variables refer to the same object in memory.

```
a = [1, 2]
b = a

print(a is b)        # True
print(a is not b)  # False
```

Strings in Python

A string (str) is a data type used to represent a sequence of characters enclosed in quotes.

In simple words:
A string is text data like names, sentences, or messages.

How to Create Strings

```
name = "Nabeela"
city = 'Hyderabad'
message = """Welcome to Python"""
```

Strings are created by enclosing characters inside single, double, or triple quotes.

Characteristics of Strings

- **Strings are ordered** → characters have positions (indexing)
- **Strings are immutable** → cannot be changed after creation

- Strings can contain letters, numbers, symbols, and spaces

Accessing Characters (Indexing)

text = "Python"

```
print(text[0])  # P
print(text[-1]) # n
```

Indexing is used to access individual characters using their position.

String Operations

1. Concatenation

```
a = "Hello"
b = "World"

print(a + " " + b)  # Hello World
```

Combining two or more strings into one.

2. Repetition

```
print("Hi " * 3)  # Hi Hi Hi
```

Repeating a string multiple times.

Common String Methods

text = " python "

```
print(text.upper())   # PYTHON
print(text.lower())   # python
print(text.strip())   # python
print(len(text))      # 8
```

- **upper()** → Converts to uppercase
- **lower()** → Converts to lowercase
- **strip()** → Removes extra spaces
- **len()** → Returns length of string

String Slicing

```
text = "Python"

print(text[0:4])  # Pyth
print(text[:3])   # Pyt
print(text[2:])   # thon
```

Slicing is used to extract a part (substring) from a string.

Strings are essential for handling and processing text data in Python programs.

Indexing in Python

Indexing is the process of accessing individual elements of a sequence (such as strings, lists, or tuples) using their position (index number).

In simple words:
Indexing helps you retrieve a specific value from data using its position.

Indexing in Strings

```
text = "Python"

print(text[0])   # P
print(text[1])   # y
print(text[5])  # n
```

In strings, indexing allows you to access each character based on its position.

Rule:

- **Index starts from 0**

Visual Representation

```
P y t h o n
0 1 2 3 4 5
```

Each element in a sequence is assigned a unique index starting from 0.

Negative Indexing

```
text = "Python"

print(text[-1])  # n
print(text[-2])  # o
```

Definition:

- Negative indexing is used to access elements from the end of the sequence.

Rule:

- -1 → Last element
- -2 → Second last element

Index Range Rule

```
text = "Python"

print(text[10])  # ❌ Error
```

Definition:
Accessing an invalid index raises an IndexError (index out of range).

Rule:
Valid index range is from:

- 0 to length - 1
- -1 to -length

Indexing in Lists

```
numbers = [10, 20, 30]

print(numbers[0])   # 10
print(numbers[-1])  # 30
```

Definition:

- Indexing works the same way for lists and other sequences.

Slicing in Python

Slicing is the process of extracting a portion (subset) of a sequence (string, list, or tuple) using a range of indices.

In simple words:
Slicing lets you cut and get a part of data.

Syntax of Slicing

```
sequence[start : stop : step]
```

Definition:

- **start** → Index where slicing begins (inclusive)
- **stop** → Index where slicing ends (exclusive)
- **step** → Interval between elements

Basic Examples

```
text = "Python"

print(text[0:4])  # Pyth
print(text[:3])   # Pyt
print(text[2:])   # thon
```

Slicing returns a new sequence containing selected elements.

Step in Slicing

```
text = "Python"
print(text[::2])  # Pto
```

The step value controls how many positions to skip.

Negative Slicing

```
text = "Python"
print(text[-4:-1])  # tho
```

You can use negative indices to slice from the end.

Reverse a String

```
text = "Python"
print(text[::-1])  # nohtyP
```

Using a negative step (-1) reverses the sequence.

String Methods (Important)

.

1. find( )

Returns the index of the first occurrence of a substring.

```
text = "python"
print(text.find("t"))  # 2
```

2. count( )

Counts how many times a substring appears.

```
text = "banana"
print(text.count("a"))  # 3
```

3. startswith( )

Checks if a string starts with a given value.

```
text = "python"
print(text.startswith("py"))  # True
```

4. endswith( )

Checks if a string ends with a given value.

```
text = "python"
print(text.endswith("on"))  # True
```

Real Example

```
text = "  data science  "

clean = text.strip().upper()
part = clean[:4]

print(part)  # DATA
```

Slicing and string methods help you extract, analyze, and manipulate parts of data efficiently in Python.

Lists in Python

A list is a data type used to store multiple items in a single variable in an ordered and changeable (mutable) way.

In simple words:
A list is a collection of values stored together.

How to Create a List

```
numbers = [1, 2, 3, 4]
names = ["Satya", "Ravi", "Anu"]
```

A list is created by placing elements inside square brackets [ ] separated by commas.

Characteristics of Lists

Definition:

- **Ordered** → items have a fixed position (index)
- **Mutable** → can be changed after creation
- **Allows duplicates**
- Can store **different data types**

```
data = [1, "Python", 3.5]
```

Accessing Elements (Indexing)

```
numbers = [10, 20, 30]

print(numbers[0])   # 10
print(numbers[-1])  # 30
```

Indexing is used to access elements using their position.

List Slicing

```
numbers = [10, 20, 30, 40]
print(numbers[1:3])  # [20, 30]
```

Slicing extracts a sub-list using a range of indices.

Common List Methods

```
numbers = [1, 2, 3]

numbers.append(4)     # Adds element at end
numbers.insert(1, 10)    # Inserts at position
numbers.remove(2)     # Removes value
numbers.pop()         # Removes last element
```

Definitions:

append( ) → Adds an item to the end
insert( ) → Adds item at a specific index
remove( ) → Removes a specific value
pop( ) → Removes and returns last item

List Operations

```
a = [1, 2]
b = [3, 4]

print(a + b)    # [1, 2, 3, 4]
print(a * 2)    # [1, 2, 1, 2]
```

Lists support operations like concatenation and repetition.

Real Example

```
students = ["Satya", "Ravi"]

students.append("Anu")
print(students)
```

Lists are widely used to store, manage, and manipulate collections of data.

Indexing in Lists

.

Indexing in lists is the process of accessing individual elements of a list using their position (index number).

In simple words:
Indexing helps you get a specific item from a list using its position.

Basic Example

```
numbers = [10, 20, 30, 40]

print(numbers[0])  # 10
print(numbers[1])  # 20
```

Each element in a list is assigned an index starting from 0.

Visual Representation

```
10  20  30  40
 0   1   2   3
```

The position of each element is called its index, which is used to access it.

Negative Indexing

```
numbers = [10, 20, 30, 40]

print(numbers[-1])  # 40
print(numbers[-2])  # 30
```

Negative indexing allows you to access elements from the end of the list.

Rule

- -1 → Last element
- -2 → Second last element

Modifying Elements

```
numbers = [10, 20, 30]

numbers[1] = 99
print(numbers)  # [10, 99, 30]
```

Lists are mutable, so you can change elements using indexing.

Index Range Rule

```
numbers = [10, 20, 30]

print(numbers[5])  # ❌ Error
```

Accessing an invalid index raises an IndexError (index out of range).

Valid index range:

- 0 to length - 1
- -1 to -length

Real Example

```
students = ["Satya", "Ravi", "Anu"]

first = students[0]
last = students[-1]

print(first, last)  # Satya Anu
```

Indexing in lists allows you to access, update, and manage elements efficiently.

Indexing is a core concept for working with lists, helping you retrieve and modify specific elements easily.

List Slicing in Python

.

List slicing is the process of extracting a portion (sub-list) from a list using index ranges.

In simple words:

List slicing helps you get a part of a list.

Syntax

```
list[start : stop : step]
```

Definition:

- start → Index where slicing begins (inclusive)
- stop → Index where slicing ends (exclusive)
- step → Number of steps to skip

Basic Examples

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])  # [20, 30, 40]
print(numbers[:3])   # [10, 20, 30]
print(numbers[2:])   # [30, 40, 50]
```

Step in Slicing

```
numbers = [10, 20, 30, 40, 50]
print(numbers[::2])  # [10, 30, 50]
```

The **step controls** how elements are skipped.

Negative Slicing

```
numbers = [10, 20, 30, 40, 50]
print(numbers[-4:-1])  # [20, 30, 40]
```

Reverse a List

```
numbers = [10, 20, 30, 40]
print(numbers[::-1])  # [40, 30, 20, 10]
```

Modifying Using Slicing

```
numbers = [10, 20, 30, 40]

numbers[1:3] = [99, 88]
print(numbers)  # [10, 99, 88, 40]
```

Slicing can be used to update multiple elements at once.

List Methods in Python

List methods are built-in functions associated with lists that help you modify, manage, and manipulate list data.

In simple words:
List methods help you work with list data easily.

Common List Methods

```
numbers = [1, 2, 3]
```

1. append()
Adds an element to the end of the list.

```
numbers.append(4)
# [1, 2, 3, 4]
```

2. insert( )
Inserts an element at a specific position.

```
numbers.insert(1, 10)
# [1, 10, 2, 3, 4]
```

3. remove( )

Removes a specific value from the list.

```
numbers.remove(2)
# [1, 10, 3, 4]
```

4. pop()

Remove and returns the last element (or specific index).

```
numbers.pop()
# removes last element
```

5. sort()

Sorts the list in ascending order.

```
numbers.sort()
```

6. reverse()

Reverses the order of elements.

```
numbers.reverse()
```

7. index()

Returns the index of a value.

```
numbers.index(10)
```

8. count ( )
Counts how many times a value appears.

```
numbers.count(1)
```

Real Example

```
students = ["Satya", "Ravi", "Anu"]

top_students = students[:2]
students.append("Kiran")

print(top_students)  # ['Satya', 'Ravi']
print(students)      # ['Satya', 'Ravi', 'Anu', 'Kiran']
```

Key Difference

Slicing → Extracts data
Methods → Modify or analyze data

List slicing and methods together allow you to ***efficiently access, modify, and manage list data in Python programs.***

Tuples in Python

A tuple is a data type used to store multiple items in a single variable in an ordered and immutable way.

In simple words:
A tuple is a collection of values that cannot be changed.

How to Create a Tuple

```
numbers = (1, 2, 3)
names = ("Satya", "Ravi", "Anu")
```

Adds anA tuple is created by placing elements inside parentheses ( ) separated by commas.

Single Element Tuple

```
single = (10,)
```

A comma is required to create a single-element tuple, otherwise it is treated as a normal value.

Characteristics of Tuples

- **Ordered** → elements have fixed positions (indexing)
- **Immutable** → cannot be changed after creation
- Allows duplicates
- Can store **different data types**

```
data = (1, "Python", 3.5)
```

Accessing Elements (Indexing)

```
numbers = (10, 20, 30)

print(numbers[0])   # 10
print(numbers[-1])  # 30
```

Indexing is used to access elements using their position.

Tuple Slicing

```
numbers = (10, 20, 30, 40)
print(numbers[1:3])  # (20, 30)
```

Slicing extracts a part of the tuple as a new tuple.

Tuple Operations

```
a = (1, 2)
b = (3, 4)

print(a + b)  # (1, 2, 3, 4)
print(a * 2)  # (1, 2, 1, 2)
```

Tuples support operations like concatenation and repetition.

Tuple Methods

```
numbers = (1, 2, 2, 3)

print(numbers.count(2))  # 2
print(numbers.index(3))  # 3
```

Definitions:

- **count()** → Counts occurrences of a value
- **index()** → Returns index of a value

Tuples are faster and more memory-efficient than lists because they are immutable.

Tuple Methods in Python

Tuple methods are built-in functions associated with tuples that allow limited operations on tuple data.

In simple words:
Tuple methods help you work with tuple data, but cannot modify it.

Important Note:

Tuples are immutable, so they have very few methods compared to lists.

Common Tuple Methods

1. count( )

Returns the number of times a specified value appears in the tuple.

```
numbers = (1, 2, 2, 3, 2)
print(numbers.count(2))  # 3
```

2. index( )

Returns the index (position) of the first occurrence of a specified value.

```
numbers = (10, 20, 30, 20)
print(numbers.index(20))  # 1
```

Additional Useful Functions

These are not tuple methods but are commonly used with tuples:

3. len()

Returns the total number of elements in the tuple.

```
numbers = (1, 2, 3)
print(len(numbers))  # 3
```

#Real Example

```
data = (1, 2, 2, 3, 4)

count_2 = data.count(2)
index_3 = data.index(3)
print(count_2, index_3)
```

Sets in Python

A set is a data type used to store multiple unique elements in an unordered collection.

In simple words:
A set is a collection of values where duplicates are not allowed.

How to create a set :

```
numbers = {1, 2, 3, 4}
names = {"Satya", "Ravi", "Anu"}
```

Empty set

```
empty = set( )
```

You must use set() to create an empty set (not {})

Characteristics of Sets:

- **Unordered** → no fixed position (no indexing)
- **Unique elements** → duplicates are removed automatically
- **Mutable** → can add or remove elements
- Cannot contain duplicate values

```
data = {1, 2, 2, 3}
print(data)  # {1, 2, 3}
```

Adding Elements

```
numbers = {1, 2}
numbers.add(3)
print(numbers)  # {1, 2, 3}
```

add( ) is used to insert a new element into a set.

Removing Elements

```
numbers = {1, 2, 3}
numbers.remove(2)
print(numbers)  # {1, 3}
```

- **remove( )** → Removes a specific element (error if not found)
- **discard( )** → Removes element safely (no error)

Set Operations

- **Union (|)** → Combines all unique elements
- **Intersection (&)** → Common elements
- **Difference (-)** → Elements in one set but not the other

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)     # Union → {1, 2, 3, 4, 5}
print(a & b)    # Intersection → {3}
print(a - b)    # Difference → {1, 2}
```

Common Set Methods

```
numbers = {1, 2, 3}

numbers.add(4)
numbers.pop()
numbers.clear()
```

- **add() → Adds element**
- **pop() → Removes a random element**
- **clear() → Removes all elements**

Sets are powerful for removing duplicates and performing mathematical operations on data.

Dictionary in Python

A dictionary (dict) is a data type used to store data in key-value pairs.

In simple words:
A dictionary stores data as pairs (key → value).

How to create a Dictionary :

```
student = {
    "name": "Satya",
    "age": 25,
    "course": "Python"
}
```

A dictionary is created using curly braces { } with key-value pairs separated by :.

Characteristics of Dictionary

- **Unordered** → elements are not accessed by index
- **Mutable** → can be changed after creation
- Keys must be **unique**
- Values can be **any data type**

Accessing Values

```
student = {"name": "Satya", "age": 25}

print(student["name"])  # Satya
```

Values are accessed using their keys, not index.

Adding / Updating Values

```
student = {"name": "Satya"}

student["age"] = 25              # Add
student["name"] = "Kumar"   # Update
```

Dictionaries are mutable, so you can add or update key-value pairs.

Removing Items

```
student = {"name": "Satya", "age": 25}

student.pop("age")
```

pop( ) removes a key-value pair using the key.

Common Dictionary Methods

```
student = {"name": "Satya", "age": 25}
```

1. keys( ) Returns all keys in the dictionary.

```
print(student.keys( ))
```

2. values( ) Returns all values in the dictionary.

```
print(student.values( ))
```

3. items( ) Returns all key-value pairs as tuples.

```
print(student.items( ))
```

4. get( ) Returns the value for a key (safe method).

```
print(student.get("name"))
```

Real Example:

```
user = {
   "username": "satya",
   "score": 95
}

print(user["username"], user["score"])
```

Dictionaries help you organize and access data efficiently using keys instead of positions.

Indexing in Dictionary

Dictionaries do not support indexing by position like lists or strings.
Instead, values are accessed using keys.

In simple words:
You don't use numbers, you use keys to access values.

Example

```
student = {"name": "Satya", "age": 25}
print(student["name"])  # Satya
```

Accessing values using keys is called key-based access (not indexing).

Safe Access

```
print(student.get("age"))  # 25
```

get( ) is used to avoid errors if the key does not exist.

Slicing in Dictionary

Dictionaries do not support slicing because they are unordered and not index-based.

Invalid Example

```
student = {"name": "Satya", "age": 25}

# print(student[0:1]) ❌ Error
```

Feature	String (str)	List (list)
Definition	Sequence of characters	Ordered collection of items
Syntax	`"hello"`	`[1, 2, 3]`
Ordered	✅ Yes	✅ Yes
Mutable	❌ No	✅ Yes
Duplicates	✅ Allowed	✅ Allowed
Indexing	✅ Yes	✅ Yes
Data Types	Characters only	Mixed allowed
Use Case	Text processing	Dynamic collections
Performance	Fast for text ops	Flexible but slightly slower
Example Access	`s[0]` → "h"	`l[0]` → `1`

Feature	Tuple (tuple)	Set (set)	Dictionary (dict)
Definition	Ordered, immutable collection	Unordered collection of unique items	Key-value pair collection
Syntax	`{1, 2, 3}`	`{1, 2, 3}`	`{"a": 1, "b": 2}`
Ordered	✅ Yes	❌ No	✅ Yes (Python 3.7+)
Mutable	✅ Yes	✅ Yes	✅ Yes
Duplicates	❌ No	❌ Not allowed	❌ Keys: No, Values: Yes
Indexing	✅ Not allowed	❌ No	✅ Yes (Access via keys)
Data Types	Mixed allowed	Mixed allowed	Keys & values can be mixed
Use Case	Fixed data (constants)	Unique items, remove duplicates	Structured data (note (mapping)
Performance	Faster than list	Fast membership testing (in)	Fast key-based lookup
Example Access	`t[0]` → `1`	Not possible	Not possible

Functions in Python

A function is a block of reusable code that performs a specific task.

In simple words:
A function is a set of instructions you can reuse whenever needed.

Why Use Functions?

Functions help to:

- Avoid code repetition
- Improve readability
- Make programs modular

Creating a Function

```
def greet():
    print("Hello!")
```

- **def** → Keyword used to define a function
- **greet** → Function name
- **()** → Parameters (optional)

Calling a Function

```
greet( )
```

Calling a function means executing the function code.

Function with Parameters

```
def greet(name):
    print("Hello", name)

greet("Satya")
```

Parameters are variables passed into a function to accept input values.

Function with Return Value

```
def add(a, b):
    return a + b

result = add(2, 3)
print(result)  # 5
```

- **return** sends a value back from the function
- The returned value can be stored in a variable

Default Parameters

```
def greet(name="Guest"):
    print("Hello", name)

greet()          # Hello Guest
greet("Satya")  # Hello Satya
```

Default parameters allow a function to use a default value if no argument is provided.

Multiple Return Values

```
def calc(a, b):
    return a + b, a - b

x, y = calc(5, 3)
print(x, y)  # 8 2
```

A function can return multiple values as a tuple.

- **Built-in functions** → Predefined (e.g., print(), len())
- **User-defined functions** → Created by the user

Types of Arguments in Python

Arguments are the values passed to a function when it is called.

In simple words:
Arguments are the inputs given to a function.

1. Positional Arguments

Arguments are passed in the same order as parameters are defined.

```
def greet(name, age):
    print(name, age)

greet("Satya", 25)
```

2. Keyword Arguments

Arguments are passed using parameter names (keys).

```
def greet(name, age):
    print(name, age)

greet(age=25, name="Satya")
```

3. Default Arguments

Parameters are given default values, used when no argument is passed.

```
def greet(name="Guest"):
    print("Hello", name)

greet()             # Hello Guest
greet("Satya")   # Hello Satya
```

4. Variable-Length Arguments (*args)

Allows a function to accept any number of positional arguments.

```
def add(*numbers):
    print(sum(numbers))

add(1, 2, 3, 4)
```

*args collects multiple values into a tuple.

5. Keyword Variable-Length Arguments (**kwargs)

Allows a function to accept any number of keyword arguments.

```
def details(**data):
    print(data)

details(name="Satya", age=25)
```

****kwargs** collects data into a dictionary.

Real Example

```
def student(name, age=20, *marks, **info):
    print(name, age)
    print(marks)
    print(info)

student("Satya", 25, 90, 95, city="Hyderabad")
```

*args collects multiple values into a tuple.

map( ) in Python

map() is a built-in function used to apply a given function to each item in an iterable (like list, tuple, etc.).

In simple words:
map() helps you apply the same operation to all elements in a sequence.

Syntax

```
map(function, iterable)
```

- **function** → The function to apply
- **iterable** → The data (list, tuple, etc.)

Basic Example

```
numbers = [1, 2, 3, 4]

def square(x):
    return x * x

result = map(square, numbers)
print(list(result))  # [1, 4, 9, 16]
```

Lambda Functions in Python

A lambda function is a small, anonymous (unnamed) function defined using the lambda keyword.

In simple words:
A lambda function is a quick, one-line function used for simple tasks.

When to Use

Use lambda when:

- Function is small and simple
- Used once
- Needed inside functions like **map(), filter(), sorted( )**

Syntax

```
lambda arguments : expression
```

Definitions:

- **arguments** → Input values
- **expression** → Single operation (result is returned automatically)

Basic Example

```
square = lambda x: x * x

print(square(4))  # 16
```

Lambda functions do not require def or return.

Without Variable (Direct Use)

```
print((lambda x: x + 5)(3))  # 8
```

Lambda functions can be used instantly without assigning to a variable.

Using with map( )

```
numbers = [1, 2, 3]

result = map(lambda x: x * 2, numbers)
print(list(result))  # [2, 4, 6]
```

Lambda is commonly used with functions like map() for quick operations.

Using with filter( )

```
numbers = [1, 2, 3, 4, 5]

result = filter(lambda x: x % 2 == 0, numbers)
print(list(result))  # [2, 4]
```

filter( ) selects elements based on a condition.

Using with sorted( )

```
data = [(1, 2), (3, 1), (5, 0)]

result = sorted(data, key=lambda x: x[1])
print(result)  # [(5, 0), (3, 1), (1, 2)]
```

Lambda helps define custom sorting logic

Key Points

Anonymous (no name)
One-line function
Only one expression allowed
Automatically returns value

filter( ) in Python

filter() is a built-in function used to select elements from an iterable based on a condition (True/False).

In simple words:
filter() helps you keep only the elements that satisfy a condition.

Syntax

```
filter(function, iterable)
```

- **function** → A condition (returns True or False)
- **iterable** → Data (list, tuple, etc.)

Basic Example

```
numbers = [1, 2, 3, 4, 5]

def is_even(x):
    return x % 2 == 0

result = filter(is_even, numbers)
print(list(result))  # [2, 4]
```

filter() returns only the elements for which the function returns True.

Using with lambda

```
numbers = [1, 2, 3, 4, 5]

result = filter(lambda x: x > 2, numbers)
print(list(result))  # [3, 4, 5]
```

Lambda is often used with filter() for short conditions.

Key Points

Selects elements based on condition
Returns a filter object
Convert to list using list()
Commonly used with lambda

Difference from map( )

map( ) → Transforms all elements
filter( ) → Selects specific elements

filter() is useful for extracting required data from a collection efficiently.

reduce() in Python

reduce() is a function used to apply a function cumulatively to the elements of an iterable, reducing it to a single value.

In simple words:
reduce() takes a list and combines all elements into one result.

Import Statement

```
from functools import reduce
```

reduce() is not built-in directly; it is available in the functools module.

Syntax

```
reduce(function, iterable)
```

- **function** → A function that takes two arguments
- **iterable** → Data (list, tuple, etc.)

How It Works

It works step-by-step:

Step 1: func(a, b) → result1
Step 2: func(result1, c) → result2
Step 3: func(result2, d) → final result

Basic Example (Sum)

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, numbers)
print(result)  # 10
```

Example (Multiplication)

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x * y, numbers)
print(result)  # 24
```

Difference (map vs filter vs reduce)

- **map()** → Transforms data
- **filter()** → Selects data
- **reduce()** → Combines data

reduce( ) is powerful for performing cumulative operations and producing a single result from a collection.

File Handling in Python

File handling is the process of creating, opening, reading, writing, and closing files in a program.

In simple words:
File handling lets you store and retrieve data from files.

Opening a File

```
file = open("data.txt", "r")
```

open() is used to open a file.

Syntax

```
open(filename, mode)
```

File Modes

Modes define how the file is used.

- "r" → Read (default)
- "w" → Write (overwrites file)
- "a" → Append (adds data)
- "x" → Create new file
- "b" → Binary mode
- "t" → Text mode

Reading a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
```

read( ) reads the entire file content.

Other Read Methods

```
file.readline()   # Reads one line
file.readlines()  # Reads all lines as a list
```

- **readline( )** → Reads one line at a time
- **readlines( )** → Reads all lines into a list

Writing to a File

```
file = open("data.txt", "w")
file.write("Hello World")
```

write( ) is used to add data to a file.

Appending to a File

```
file = open("data.txt", "a")
file.write("New Line")
```

append mode adds data without deleting existing content.

Closing a File

```
file.close( )
```

close( ) releases the file and frees system resources.

Using with (Best Practice)

```
with open("data.txt", "r") as file:
    content = file.read( )
    print(content)
```

with automatically handles opening and closing files safely.

Key Points

Always close files (or use with)
Choose correct mode (r, w, a)
Prevent data loss when writing

File handling is essential for storing, managing, and processing data outside your program.

Exception Handling in Python

Exception handling is the process of handling errors (exceptions) that occur during program execution to prevent the program from crashing.

In simple words:
Exception handling helps you manage errors safely without stopping the program.

What is an Exception?

An exception is an error that occurs during the execution of a program.

```
print(10 / 0)  # ❌ ZeroDivisionError
```

Basic Structure

```
try:
    # risky code
except:
    # error handling code
```

File Modes

- **try** → Code that may cause an error
- **except** → Code that handles the error

Example:

```
try:
    x = 10 / 0
except:
    print("Error occurred")
```

Handling Specific Exceptions

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number")
```

You can handle specific types of errors.

Multiple Exceptions

```
try:
    x = int("abc")
    y = 10 / 0
except ValueError:
    print("Value Error")
except ZeroDivisionError:
    print("Division Error")
```

else Block:

```
try:
    x = 10 / 2
except:
    print("Error")
else:
    print("Success")
```

else runs only if no exception occurs.

finally Block

```
try:
    x = 10 / 2
except:
    print("Error")
finally:
    print("Always runs")
```

finally runs no matter what (error or not).

Raising Exceptions

```
x = -1

if x < 0:
    raise ValueError("Negative value not allowed")
```

raise is used to manually trigger an exception.

Real Example

```
try:
    num = int(input("Enter number: "))
    print(10 / num)
except ValueError:
    print("Enter valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Key Points

- Prevents program crash
- Handles runtime errors
- Improves program reliability
- Use specific exceptions when possible

Exception handling makes your program robust, safe, and user-friendly by managing errors effectively.